

Performance and Scalability

Main Issue

To keep replicas consistent, all **conflicting operations** should be done in the same order everywhere.

Conflicting Operations

Read-write conflict:

A read and a write operation happen at the same time.

Write-write conflict:

Two write operations happen at the same time.

Issue

Global ordering is costly and can reduce scalability.

A possible solution is to **weaken consistency requirements to avoid global synchronization**.

Data-Centric Consistency Models

Consistency Model

A consistency model is a contract between a distributed data store and processes.

It defines what results read and write operations should return when operations happen concurrently.

Essential

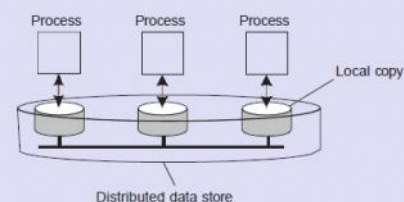
A data store is a distributed collection of storages.

Each process may access a local copy of the data.

Short Meaning

Consistency models define how replicated data should behave when many processes read and write at the same time.

A data store is a distributed collection of storages:



Continuous Consistency

Consistency is not always only **consistent** or **not consistent**.

We can talk about a **degree of consistency**.

Replicas May Differ In

- **Numerical value**: replicas may store different values.
- **Staleness**: one replica may have older data than another.
- **Update operations**: replicas may differ in the number or order of updates.

Sequential Consistency



The result must look as if all operations were executed in **one sequential order**.

Also, each process's own operations must appear in the order written in its program.

(b)

(a) A sequentially consistent data store. (b) A data store that is not sequentially consistent

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

- Bir process önce b, sonra a görmüş.
- Başka bir process önce a, sonra b görmüş.

Bu durumda herkes için geçerli **tek bir global sıra** kurulamıyor.

Bu yüzden **not sequentially consistent**.

Causal Consistency

Causal consistency only cares about **related write operations**.
If one write may affect another write, all processes **must** see them in the **same order**.

Example

- 1. **P1 writes:** $x = a$
 - 2. **P2 reads:** $x = a$
 - 3. Then **P2 writes:** $x = b$
- Here, b depends on a .
So, all processes must see:
first $x = a$, then $x = b$

Concurrent Writes

If two writes are independent, they do not have to be seen in the same order.
Example:

- **P1 writes:** $x = a$
- **P2 writes:** $x = b$

If they do not know about each other, they are concurrent.
So, one process may see:
 $a \rightarrow b$
Another process may see:
 $b \rightarrow a$

(a) A violation of a causally-consistent store. (b) A correct sequence of events in a causally-consistent store

P1: W(x)a	P1: W(x)a
P2: R(x)a W(x)b	P2: W(x)b
P3: R(x)b R(x)a	P3: R(x)b R(x)a
P4: R(x)a R(x)b	P4: R(x)a R(x)b

W(x)b depends on W(x)a.
Because **P2 reads a first**, then writes **b**.
So, all processes must see the same order:
first a, then b
But **P3 sees b before a**.
This breaks causal consistency.

W(x)a and W(x)b are independent writes.
P2 does not read **a** before writing **b**.
So, different processes may see different orders:

- **P3:** $b \rightarrow a$
- **P4:** $a \rightarrow b$

This is allowed in causal consistency.

Grouping Operations

Definition !
Operations can be grouped by using **locks**.

- Accesses to locks are sequentially consistent.
- A lock cannot be used until previous writes are completed everywhere.
- Data cannot be accessed until previous lock operations are completed.

Güncel veri görünürlüğü, tüm sisteme anında yayılmıyor; ilgili lock üzerinden kontrol ediliyor.

Basic Idea
We do not need every single read and write to be immediately visible.
We only need the **effect of the whole operation group** to be visible.

Entry Consistency

A valid event sequence for entry consistency					
P1:	L(x)	W(x)a	L(y)	W(y)b	U(x) U(y)
P2:			L(x)	R(x)a	R(y) NIL
P3:				L(y)	R(y)b

A process locks data before using it and unlocks it after finishing.
Example:

- **L(x):** lock x
- **W(x)a:** write a to x
- **U(x):** unlock x

Other processes can safely read the data after the lock is released.

P2 sees $x = a$ because it locks x.
P2 does not have to see $y = b$ because it did not lock y.
P3 sees $y = b$ because it locks y.

Consistency for Mobile Users

Example

A mobile user may access a distributed database from different locations.

At **location A**, the user reads and updates data.

At **location B**, the user continues working, but may connect to a different replica.

This can cause inconsistencies:

- updates from A may not have reached B yet
- B may have newer data than A
- updates at B may conflict with updates at A

Main Idea

The user wants the data at B to look the same as they left it at A.

So, the database should appear consistent **to that user**.

Basic Architecture

Mobile User and Replicas

A mobile client can move to another location and connect to another replica.

The distributed database has many replicas connected through a wide-area network.

Client-Centric Consistency

Replicas need to maintain consistency from the **client's point of view**.

The goal is not always global consistency for everyone.

The goal is that the moving user sees their **own reads and updates correctly**. **Not uygulamasına not yazmak gibi o güncel olsun yeter , çünkü client centric amac kendi read ve updatelerini güncel görmek**

Client-Centric Consistency: Notation

Notation

W1(x2) means:

Process **P1** writes data item **x** and creates version **x2**.

W1(xi ; xj) means:

Process **P1** creates version **xj** based on previous version **xi**. bağımlı

W1(xi | xj) means:

Process **P1** creates version **xj** concurrently with version **xi**. bağımsız

! Monotonic Reads : Once a user sees a version, they should never see an older version later.

Definition

If a process reads a value of data item **x**, later reads of **x** must return the same value or a newer value. It should not go back to an older value.

Main Idea

A mobile user may connect to different replicas.

Even then, the user should not see old data after seeing newer data.

Examples

If you read your email from different mail servers, each new server should fetch at least the updates from the server you used before.

Monotonic Writes

Definition

Monotonic writes means that write operations from the same process must be completed in the same order everywhere.

If a process writes a new version, its previous write must be completed first.

Short meaning:

A user's writes should never be reordered.

(a) A monotonic-write consistent data store. (b) A data store that does not provide monotonic-write consistency. (c) Again, no consistency as $WS(x_1|x_2)$ and thus also $WS(x_1|x_3)$. (d) Consistent as $WS(x_1,x_3)$ although x_1 has apparently overwritten x_2 .

L1: $W_1(x_1)$

L2: $W_2(x_1|x_2)$ $W_1(x_2|x_1)$

(a)

L1: $W_1(x_1)$

L2: $W_2(x_1|x_2)$ $W_1(x_1|x_2)$

(b)

L1: $W_1(x_1)$

L2: $W_2(x_1|x_2)$ $W_1(x_2|x_1)$

(c)

L1: $W_1(x_1)$

L2: $W_2(x_1|x_2)$ $W_1(x_1|x_2)$

(d)

(b) Not Consistent

P1 writes **x1**.
Then P2 creates **x2** concurrently with **x1**.
Then P1 creates **x3** concurrently with **x1**.
There is no clear version order like:
x1 → x2 → x3
So, monotonic-write consistency is not guaranteed.

(a) Correct

P1 first writes **x1**.
Then P2 creates **x2** based on **x1**:
x1 → x2
Then P1 creates **x3** based on **x2**:
x1 → x2 → x3
This is consistent because the versions form a clear order.

(c) Not Consistent

P2 creates **x2** concurrently with **x1**.
Then P1 creates **x3** based on **x2**.
But because **x2** was not clearly ordered after **x1**, **x3** is also not clearly ordered after **x1**.
So, this is not consistent.

(d) Consistent

P1 writes **x1**.
Then P2 creates **x2** concurrently on **x1**.
Then P1 creates **x3** based on **x1**.
Even if **x3** seems to overwrite **x2**, P1's own write order is still preserved.
So, this is consistent for monotonic writes.

Monotonic Writes: Examples

Program Update Example

If a program is updated on server **S2**, all required previous components must also be available on **S2**.

For example, a new program version may need old libraries or previous files.

So, the previous writes must reach **S2** before the new update is installed.

Replicated File Version Example

Replicated files must keep their versions in the correct order everywhere.

If version **x2** is based on version **x1**, then **x1** must be propagated before **x2** is installed.

(a) A data store that provides read-your-writes consistency. (b) A data store that does not.

L1: $W_1(x_1)$

L2: $W_2(x_1, x_2)$ $R_1(x_2)$

(a)

L1: $W_1(x_1)$

L2: $W_2(x_1, x_2)$ $R_1(x_2)$

(b)

Read Your Writes

Definition

After a process writes a value, its next read must show that write.
So, a user should always see **their own updates**.

Example (a) — Correct

The process writes **x1**.
Then it later reads **x2**, which includes the update based on **x1**.
So, the user sees the effect of their own write.

Example (b) — Not Correct

The process writes **x1**.
But later, it reads **x2** without seeing the effect of its own write.
So, read-your-writes consistency is broken.

Web Page Example

If you update your web page, your browser should show the newest version.
It should not show an old cached version.

Replica Placement

Essence

Replica placement means choosing the best **K locations** from **N possible locations**.
The goal is to place replicas close to clients and reduce latency.

Approaches

Minimum average distance:

Choose locations where the average distance to clients is minimal.
This is good but computationally expensive.

Autonomous system based:

Place replicas in the largest autonomous systems and choose well-connected hosts.
This is also computationally expensive.

Geometric space based:

Put nodes in a geometric space where distance represents latency.
Choose dense regions and place replicas there.
This is computationally cheaper.

Content Replication

! Replica Types

Permanent replicas:

A machine always stores a replica.

Server-initiated replicas:

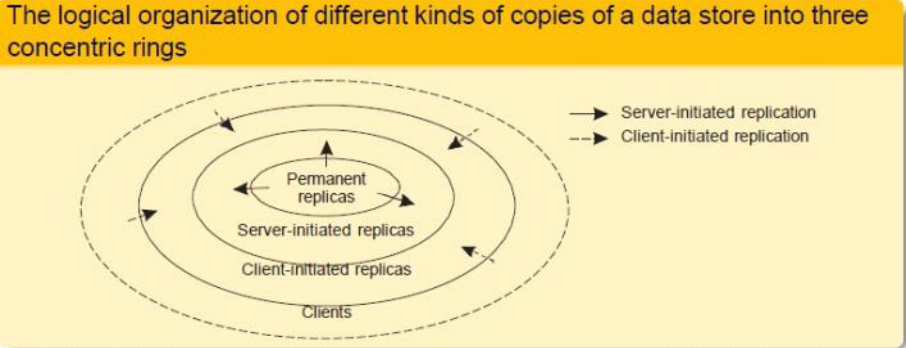
A server dynamically creates a replica when another server needs it.

Client-initiated replicas:

A client dynamically creates a local copy.
This is usually called **client cache**.

Short Meaning

Replica placement decides **where replicas should be placed**.
Content replication decides **what kind of replicas are created**.



Content Replication

Three Logical Rings

Different copies of a data store can be organized in three layers.

Permanent replicas:

These are the main replicas.
They are always available and stay in fixed servers.

Server-initiated replicas:

These are created dynamically by servers.
They are placed closer to clients when demand increases.

Client-initiated replicas:

These are created by clients.
They are usually temporary caches near the client.

Short Meaning

Permanent replicas are the core copies.
Server replicas help reduce load and latency.
Client replicas are local cached copies for faster access.

Content Distribution

Main Idea

In a client-server system, updates can be distributed in different ways.

Approaches

Notification / invalidation:

Only a message is sent to say that an update happened. just warning , no data changing
This is often used for caches.

Passive replication:

The **updated data** is transferred from one copy to another.

Active replication:

The **update operation** itself is sent to other copies.

Note :There is no single best method.

The choice depends on available bandwidth and the read/write ratio.

Update’ler kopyalara nasıl dağıtılır?

Content Distribution: Client/Server System

Push-Based Updates

Push-based update is a **server-initiated** approach.
The server sends updates to clients, even if the clients did not ask for them.

Server mı gönderir, client mı ister?

Pull-Based Updates

! Pull-based update is a **client-initiated** approach.
The client asks the server for updates.

Push vs Pull		
Issue	Push-Based	Pull-Based
State at server	List of client caches	None
Messages exchanged	Update / possibly fetch update	Poll and update
Client response time	Immediate or fetch-update time	Fetch-update time

Content Distribution: Leases!

Main Idea

A system can switch between **push** and **pull** using **leases**.

A **lease** is a temporary contract.

The server promises to push updates to the client until the lease expires.

Lease varken:
Server client'a
update push
eder.

Adaptive Leases

The lease time can change depending on the system.

Age-based leases:

If an object has not changed for a long time, it gets a longer lease.

Renewal-frequency based leases:

If a client often requests the same object, it gets a longer lease for that object.

State-based leases:

If the server is heavily loaded, lease times become shorter.

Short Meaning

Leases help balance push and pull updates.

The server pushes updates only for a limited time.

Primary-Based Protocols

Primary-Backup Protocol

In this protocol, one replica is the **primary server** for a data item.

All write requests are sent to the primary server.

Write Process

1. Client sends a write request.
2. The request is forwarded to the primary server.
3. The primary updates the backup replicas.
4. Backups acknowledge the update.
5. The write is completed.

Read Process

A client can send a read request and receive the data from a replica.

Short Meaning

The primary server controls writes.

Backup servers keep copies for fault tolerance.

This is common in distributed databases and file systems.

Primary-Based Protocols: Local Writes

In local-write protocols, the **primary copy** of a data item can be moved closer to the client.

This allows the client to write locally, which is useful for mobile or disconnected work.

After the client finishes the update, the changes are sent to backup replicas.

Short meaning:

Local writes move the primary copy near the user, so updates can be done locally and synchronized later.

To read from or write to the database, you must receive confirmation from a certain number of replicas.

Replicated-Write Protocols

In replicated-write protocols, there is no single primary server. Read and write operations use voting among replicas.

Read quorum (NR): number of replicas needed for a read.

Write quorum (NW): number of replicas needed for a write.

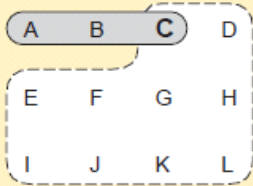
Read and write quorums should overlap, so a read can see the latest write.

ROWA: Read one, write all.

Reads are cheap, but writes are expensive.

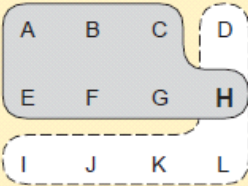
This is known as a **quorum**.

Three examples of the voting algorithm. (a) A correct choice of read and write set. (b) A choice that may lead to write-write conflicts. (c) A correct choice, known as ROWA (read one, write all)



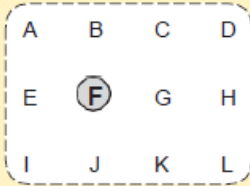
NR = 3, NW = 10

3 + 10 > 12
10 > 12/2
correct set



NR = 7, NW = 6

7+6>12
6 > 12/2
may lead write-write
conflict



NR = 1, NW = 12

12+1 > 13
12 > 13/2
Read one , write all - correct set
known as **ROWA**

NR + NW > N = (prevent read-write conflicts)

NW > N / 2 = (prevent write-write conflicts)